

Full-Grid Neural Reception for Coded 2×2 MIMO-OFDM Across 3GPP TDL Channels

Minh Phuc Nguyen

Department of Electrical, Computer, and Biomedical Engineering

Union College

Schenectady, NY 12308 USA

nguyenp@union.edu

Abstract—This paper represents the development and evaluation of an LS-assisted full-grid residual convolutional receiver for a coded 2×2 MIMO-OFDM link. The simulated link uses 3GPP TDL channels and 5G LDPC coding with customized resource grid and pilot layout. We compare one frozen neural checkpoint with a conventional LS, interpolation, LMMSE, and APP receiver across 45 channel environments. The study asks where the two receivers behave differently and how that pattern changes with channel conditions. The evaluation does not identify a uniform winner: the conventional chain has lower observed error rates at the low and highest tested SNRs, while the neural receiver is generally lower through the middle of the sweep. We focus on the construction process, comparisons and visible failures.

Index Terms—neural receiver, MIMO-OFDM, link-level simulation, LS-assisted detection, tapped-delay-line channels

I. INTRODUCTION AND STUDY QUESTIONS

Multiple-input multiple-output orthogonal frequency-division multiplexing (MIMO-OFDM) combines spatial multiplexing with robustness to frequency-selective propagation and is a central physical-layer technology in contemporary broadband wireless systems. In a coded MIMO-OFDM link, however, reliable reception requires more than the recovery of individual constellation symbols. A practical receiver must perform a series of steps, from estimating a time- and frequency-varying channel from a limited set of pilots, interpolating that estimate across the resource grid, separating simultaneously transmitted spatial streams, producing reliable soft-bit information, to passing those values to a channel decoder [1], [2]. Errors introduced during channel estimation or equalization can propagate through the demapper and ultimately determine the decoded bit-error rate (BER) and frame-error rate (FER).

Conventional receivers address these tasks through a sequence of individually designed blocks. Least-squares (LS) channel estimation is attractive because of its simplicity and limited assumptions, but its estimates can be sensitive to noise and to the density and placement of pilot symbols [1], [3]. These issues become more visible in MIMO links with frequency selectivity, mobility, sparse pilot observations, and imperfect knowledge of the operating environment.

Neural receivers provide a different way to organize the same inference problem. A learned model may replace one conventional stage or jointly perform several receiver operations. Fully convolutional receivers are particularly natural for OFDM because the received resource grid has local and long-range

structure across time, frequency, antennas, pilot positions, and data positions. The DeepRx fully convolutional deep-learning receiver demonstrated that convolutional processing over an OFDM grid can jointly recover coded-bit information without retaining all of the conventional receiver boundaries [4], [5]. Subsequent work extended full-grid reception toward MIMO and multi-user settings, flexible resource allocations, fifth-generation (5G) New Radio (NR) compatibility, real-time operation, and model-driven combinations of learned and analytical processing [6]–[8].

Adaptation is another unresolved part of neural reception. Offline training can produce a strong receiver under the channel distribution represented in its training data, but a frozen network may lose that advantage when propagation, mobility, antenna configuration, or transmission parameters change [9].

This paper takes the narrow evaluation-oriented approach. We construct a custom, NR-inspired coded 2×2 MIMO-OFDM link using channel profiles drawn from the Third Generation Partnership Project (3GPP) Technical Report (TR) 38.901 [10]. The implementation uses Sionna for differentiable communication components [11].

Two receiver paths are compared. The neural path shares the fast Fourier transform (FFT), LS/interpolation front end, and low-density parity-check (LDPC) decoder, but replaces the conventional equalization and soft-demapping core with a residual convolutional neural network (CNN).

The study is organized around two questions:

- 1) Under which tested conditions does the frozen LS-assisted convolutional receiver have a lower observed BER or FER than the LS, interpolation, linear minimum mean-square error (LMMSE), and a posteriori probability (APP) baseline?
- 2) How consistent is the receiver comparison across signal-to-noise ratio (SNR), tapped-delay-line (TDL) profile, root-mean-square (RMS) delay spread, and terminal speed, and how do the absolute BER and FER values affect the interpretation of relative improvements?

II. SIMULATED LINK AND RECEIVER PATHS

A. Fixed Link Configuration

We simulate one transmitter with two antennas and two spatial streams, and one receiver with two antennas. Table I

TABLE I
FIXED SIMULATED LINK CONFIGURATION.

Setting	Value
Transmitter / receiver	One node with two antennas / one node with two antennas
Spatial streams	Two
Resource grid	14 OFDM symbols $\times$ 64 subcarriers
Null carriers	No direct-current (DC) null or guard carriers
Subcarrier spacing	15 kHz
Sampling rate	960 kHz
Cyclic prefix (CP)	16 samples (16.67 μ s)
Full 14-symbol interval	Approximately 1.167 ms
Modulation	Normalized 16-point quadrature amplitude modulation (16-QAM)
Channel code	5G LDPC, nominal rate 1/2
Decoder iterations	20
Pilot-symbol indices	2 and 11 (zero-based)
Nonzero pilots / stream	64
Data resource elements (REs) / stream	768
Carrier frequency	3.5 GHz
Channel family	3GPP TDL-A through TDL-E
Simulation-SNR points	0 dB to 20 dB in 2 dB steps

lists the fixed link settings. The transmitter applies a rate-1/2 5G LDPC encoder, maps four coded bits to each normalized 16-point quadrature amplitude modulation (16-QAM) symbol, and places the two streams on a custom OFDM resource grid. We use the Sionna implementations of the mapper, LDPC code, OFDM blocks, and receiver components [11]. The LDPC configuration follows the coding structure in [12], but the complete link is NR-inspired rather than NR-compliant.

The grid uses $N_{sc} = 64$ subcarriers with spacing $\Delta f = 15$ kHz. Its sample rate is

$$f_s = N_{sc}\Delta f = 64(15 \text{ kHz}) = 960 \text{ kHz}. \quad (1)$$

The useful OFDM-symbol duration is $1/\Delta f = 66.67 \mu\text{s}$, and one sample lasts $1.0417 \mu\text{s}$. The 16-sample cyclic prefix therefore lasts $16.67 \mu\text{s}$. One complete custom OFDM symbol occupies about $83.33 \mu\text{s}$, so the 14-symbol interval lasts about 1.167 ms. This timing includes the cyclic prefix on every symbol. We do not treat the interval as a standard NR slot because the FFT size, carrier allocation, and cyclic prefix are custom.

B. Resource Grid and Pilot Allocation

We reserve OFDM-symbol indices 2 and 11 as pilot regions for both streams. Each region spans all 64 subcarriers. A stream therefore reserves 128 resource elements (REs) across the two pilot symbols. The two streams use alternating subcarriers inside those regions. Stream 1 sends a unit pilot on one set of subcarriers and zero on the positions used by Stream 2; Stream 2 uses the complementary set. Each stream sends 64 nonzero unit pilots in total. This mapping avoids cross-stream pilot collisions in the simulated point-to-point link. It is a custom pilot pattern, not a complete NR demodulation reference signal (DMRS) procedure.

Neither stream carries data in the two reserved pilot symbols. The remaining 12 symbols give

$$N_{\text{data}} = 12 \times 64 = 768 \quad (2)$$

data REs per stream. With two streams and four coded bits per 16-QAM symbol, one 14-symbol interval carries

$$n = 2 \times 768 \times 4 = 6144 \quad (3)$$

coded bits. The nominal rate-1/2 LDPC code maps $k = 3072$ information bits to these 6144 coded bits.

C. Time-Domain Channel and Grid Notation

After resource-grid mapping, the OFDM modulator applies an inverse fast Fourier transform (IFFT) and adds the cyclic prefix. The resulting two-antenna waveform passes through a normalized time-domain TDL channel, followed by additive white Gaussian noise (AWGN). We use TDL profiles from 3GPP TR 38.901 at a carrier frequency of 3.5 GHz [10]. The evaluation section defines the tested profile, delay-spread, and speed matrix.

For receiver processing, let l and q denote the OFDM-symbol and subcarrier indices. We write the post-FFT grid in the compact form

$$\mathbf{y}[l, q] = \mathbf{H}[l, q]\mathbf{x}[l, q] + \mathbf{n}[l, q], \quad (4)$$

where $\mathbf{x}[l, q] \in \mathbb{C}^2$ contains the two transmitted symbols, $\mathbf{y}[l, q] \in \mathbb{C}^2$ contains the two receive-antenna samples, and $\mathbf{H}[l, q] \in \mathbb{C}^{2 \times 2}$ describes the four links. We use (4) to define the grid dimensions. The simulator applies the channel to the time-domain waveform. Under motion, the FFT output can also contain intercarrier interference that the compact equation does not show explicitly.

D. Shared Receiver Front End

Figure 1 shows where the two receiver paths share processing and where they split. Both paths start from the same two-antenna received grid. Both use an LS channel estimator with linear interpolation. For each receive-antenna and stream pair, the estimator produces a channel estimate and an error-variance value over the full grid. The four interpolated channel estimates and their error variances feed the implemented LMMSE path and the neural feature constructor. Both paths also use the same hard-output LDPC decoder after producing coded-bit soft values.

E. Receiver Branches

The implemented classical path passes the received grid, interpolated LS estimates, estimator error variance, and noise power to an LMMSE equalizer. An APP demapper converts the equalized 16-QAM symbols and their effective noise variances into coded-bit soft values. The LDPC decoder then returns information bits for BER and FER measurement.

The neural path builds a 27-channel real-valued tensor from the received in-phase/quadrature (I/Q) grid, LS estimates, error variances, noise power, pilot and data masks, pilot values, and time and frequency coordinates. A residual CNN processes

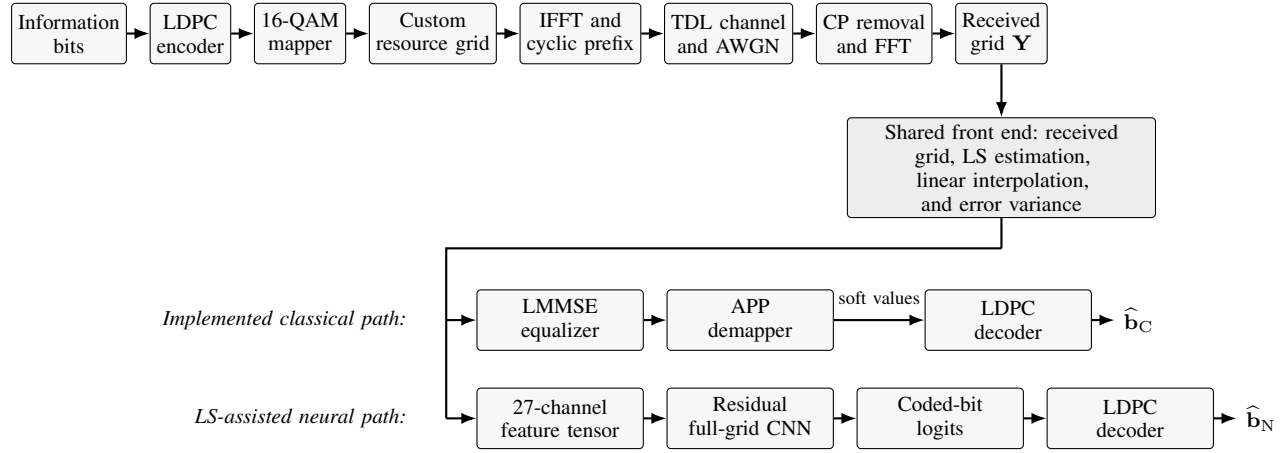


Fig. 1. Simulated link and receiver paths. The shared front end keeps the received grid and computes LS channel estimates and error variances by linear interpolation. The neural branch uses these quantities as input and replaces only the implemented LMMSE-equalization and APP-demapping core.

the full 14×64 grid and outputs one coded-bit logit for each data-bit position. After data-position masking and flattening, the logits enter the same LDPC decoder used by the classical path. The CNN therefore replaces the implemented LMMSE equalizer and APP demapper. It does not replace LS estimation, linear interpolation, OFDM demodulation, or LDPC decoding.

F. Simulation-SNR Definition

The horizontal variable in the experiments is the program's simulation-SNR parameter. For a requested value SNR_{dB} , the code sets the AWGN variance to

$$N_0 = 10^{-\text{SNR}_{\text{dB}}/10}. \quad (5)$$

The QAM mapper uses unit average constellation energy, and the time-domain channel is normalized before AWGN is added. We do not label this quantity as the ratio of energy per information bit to noise power spectral density (E_b/N_0). The conversion in (5) does not explicitly include the code rate, pilot overhead, or number of spatial streams.

III. NEURAL RECEIVER IMPLEMENTATION AND DEVELOPMENT

Our implementation keeps the classical OFDM front end and the LDPC decoder. It replaces only the implemented LMMSE equalizer and APP demapper. The model also receives the output of the LS channel-estimation path.

A. Input Tensor Construction

For a batch of size B , the input to the CNN has shape

$$\mathbf{X} \in \mathbb{R}^{B \times 27 \times 14 \times 64}. \quad (6)$$

The last two dimensions follow the OFDM grid: 14 OFDM symbols and 64 subcarriers. The 27 channels combine the received signal with known transmitter information and estimates from the classical front end. Table II lists the complete construction.

TABLE II
CHANNELS IN THE NEURAL-RECEIVER INPUT TENSOR

Feature group	Channels
Real part of the received grid, one per receive antenna	2
Imaginary part of the received grid, one per receive antenna	2
Log noise variance	1
Pilot-position masks, one per transmit stream	2
Data-position masks, one per transmit stream	2
Real part of known pilot values, one per stream	2
Imaginary part of known pilot values, one per stream	2
Normalized OFDM-symbol coordinate	1
Normalized subcarrier coordinate	1
Real part of the LS channel estimate, one per antenna pair	4
Imaginary part of the LS channel estimate, one per antenna pair	4
Log LS error variance, one per antenna pair	4
Total	27

The construction can be summarized as

$$\mathbf{X} = \text{cat}_{\text{ch}} \left(\Re\{\mathbf{Y}\}, \Im\{\mathbf{Y}\}, \log N_0, \mathbf{M}_p, \mathbf{M}_d, \Re\{\mathbf{P}\}, \Im\{\mathbf{P}\}, \mathbf{T}, \mathbf{F}, \Re\{\widehat{\mathbf{H}}\}, \Im\{\widehat{\mathbf{H}}\}, \log \mathbf{V}_e \right). \quad (7)$$

Here, $\mathbf{Y}$ is the complex received grid, $\mathbf{M}_p$ and $\mathbf{M}_d$ are the pilot and data masks, and $\mathbf{P}$ contains the known pilot values. The coordinate planes $\mathbf{T}$ and $\mathbf{F}$ run from -1 to 1 over the time and frequency axes. The tensors $\widehat{\mathbf{H}}$ and $\mathbf{V}_e$ are the interpolated LS channel estimate and its error variance. A small numerical floor is applied before the error variance is mapped to the logarithmic domain.

This design gives the CNN both the raw received samples and structured side information. The masks tell the model which resource elements have known symbols and which contribute coded bits. The pilot-value channels matter because the two streams use different pilot values at their pilot positions. The LS channels give an explicit estimate for every receive-transmit antenna pair. As a result, the neural path still depends on LS estimation and linear interpolation.

B. Residual CNN

The network is a fully convolutional residual CNN. It first maps the 27 input channels to 96 hidden channels using a 3×3 convolution and a rectified linear unit (ReLU) activation. Six residual blocks then operate at the same width. Each block contains two 3×3 convolutions, with a ReLU after the first convolution and another ReLU after the residual addition. This follows the residual learning principle in [13]. Padding preserves the 14×64 grid at every layer.

The frequency-dilation factors of the six blocks are

$$(d_1, d_2, d_3, d_4, d_5, d_6) = (1, 2, 4, 4, 2, 1), \quad (8)$$

while the time-dilation factor is one in every block. Dilated convolutions increase the frequency context without pooling or reducing the grid resolution [14]. If $\mathbf{Z}_\ell$ is the input of block ℓ , its computation is

$$\mathbf{Z}_{\ell+1} = \text{ReLU}(\mathbf{Z}_\ell + \mathcal{C}_{\ell,2}[\text{ReLU}(\mathcal{C}_{\ell,1}[\mathbf{Z}_\ell])]), \quad (9)$$

where $\mathcal{C}_{\ell,1}$ and $\mathcal{C}_{\ell,2}$ denote the two convolutions in the block. A final 1×1 convolution maps the 96 hidden channels to eight output channels. These eight values correspond to four coded bits for each of the two 16-QAM streams at a grid position.

Including convolutional biases, the number of trainable parameters is

$$\begin{aligned} & (27 \cdot 96 \cdot 3 \cdot 3 + 96) \\ & + 12(96 \cdot 96 \cdot 3 \cdot 3 + 96) + (96 \cdot 8 + 8) = 1,020,680. \end{aligned} \quad (10)$$

C. Logit Ordering and Decoder Interface

The CNN output has shape

$$\mathbf{L}_{\text{grid}} \in \mathbb{R}^{B \times 8 \times 14 \times 64}. \quad (11)$$

The implementation reshapes and permutes this tensor in two steps:

$$\begin{aligned} [B, 8, 14, 64] & \longrightarrow [B, 2, 4, 14, 64] \\ & \longrightarrow [B, 2, 14, 64, 4]. \end{aligned} \quad (12)$$

The first stream dimension separates the two spatial layers, and the last dimension holds the four bits of one 16-QAM symbol. Applying the data mask selects the 1536 data resource elements across both streams:

$$[B, 2, 14, 64, 4] \xrightarrow{M_d} [B, 1536, 4] \longrightarrow [B, 6144]. \quad (13)$$

Training uses binary cross-entropy (BCE) with logits. For target bit $b_i \in \{0, 1\}$ and network output ℓ_i , the batch loss is

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^N [b_i \log \sigma(\ell_i) + (1 - b_i) \log(1 - \sigma(\ell_i))], \quad (14)$$

where $\sigma(\cdot)$ is the sigmoid function. Under this convention, a positive output favors bit one. The decoder consumes these real-valued outputs in place of the classical demapper values.

D. Evaluation Setup

We evaluated one trained neural receiver against the receiver described in Section II. The neural model remained fixed throughout the experiment and we did not retrain or fine-tune it when the channel conditions changed. Both receivers used the same coded 2×2 MIMO-OFDM link and the same LDPC decoder, so the comparison isolates the difference between their detection and soft-demapping stages.

We tested five TDL channel profiles, three delay spreads (100 ns, 300 ns, and 1,000 ns), and three terminal speeds (0 m s^{-1} , 10 m s^{-1} , and 30 m s^{-1}). Their Cartesian product produced 45 channel environments. Each environment was evaluated from 0 to 20 dB simulation SNR in 2 dB steps. This matrix lets us examine whether the receiver comparison changes with multipath profile, frequency selectivity, mobility, or signal quality instead of relying on one favorable channel setting.

For every environment and SNR point, we ran simulations processed more than ten million coded bits, corresponding to about five million information bits after accounting for the code rate. The two receivers faced equivalent simulated inputs.

We measured BER and FER after LDPC decoding. BER captures the fraction of information bits recovered incorrectly, while FER records whether a complete transmission contained at least one information-bit error. BER and FER are presented separately because a receiver can produce fewer total bit errors without necessarily producing fewer failed frames.

IV. RESULTS

A. Reference-Condition Comparison

We first compare the receivers under TDL-A with a 300 ns RMS delay spread and a terminal speed of 10 m s^{-1} . At 3.5 GHz, the maximum Doppler frequency associated with 10 m s^{-1} is approximately 117 Hz, or less than 0.8% of the 15 kHz subcarrier spacing. The $16.67 \mu\text{s}$ cyclic prefix is also much longer than the specified RMS delay spread. Large intercarrier and intersymbol interference should therefore not be the main sources of error in this particular comparison.

Figure 2 compares two different solutions to these remaining inference problems. The classical branch applies an explicit noise-dependent LMMSE equalizer to each resource element and then computes APP soft values using an effective-noise model. The neural branch receives the same interpolated LS estimate and error variance, but also processes the raw received grid with a receptive field covering all 14 OFDM symbols and up to 59 subcarrier positions. It is a comparison between local analytical soft detection and learned contextual soft detection downstream of that estimate.

Panel (a) does not show a fixed advantage for either approach. The clearest neural separation occurs at 16 dB, where the BER decreases from 3.184×10^{-3} to 2.427×10^{-3} , a relative reduction of approximately 24%. FER decreases from 3.972×10^{-2} to 2.805×10^{-2} , or approximately 29%. The neural advantage is therefore concentrated around the decoding waterfall rather than increasing monotonically with SNR.

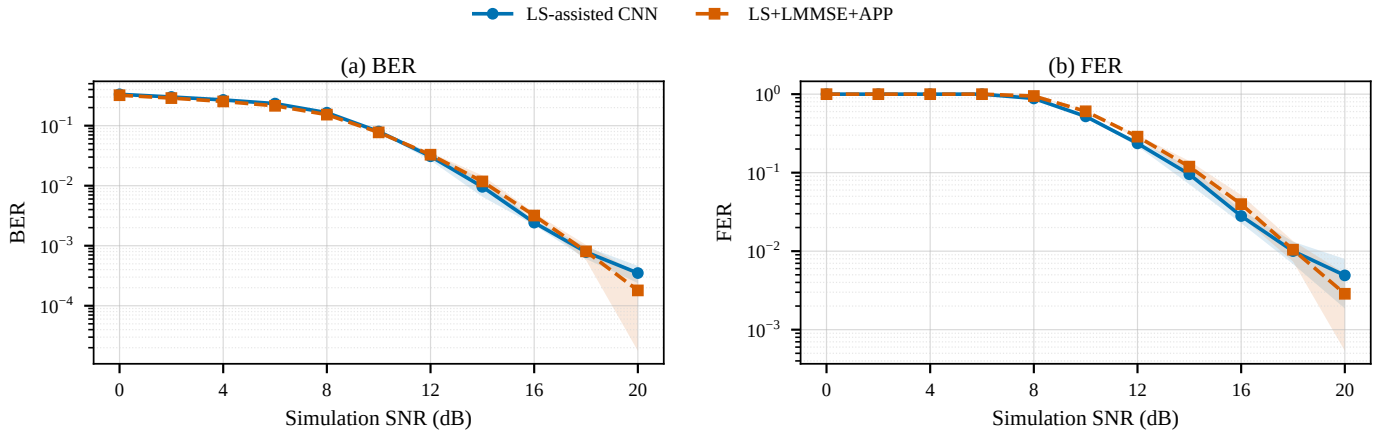


Fig. 2. Mean post-LDPC BER and FER for the TDL-A reference environment with a 300 ns RMS delay spread and a fixed speed of 10 m s^{-1} . Curves show means across evaluation seeds 1000, 1001, and 1002; shaded regions show Student- t 95% intervals. With only three seeds, the intervals provide descriptive uncertainty rather than evidence of statistical significance.

At low SNR, the LMMSE branch has a strong analytical advantage: noise power and channel-estimation uncertainty appear explicitly in its regularized inverse. We trained the neural receiver at 16–22 dB, so operation from 0 through 10 dB is extrapolation for the neural receiver. However, it is not evidence that convolutional reception is fundamentally unsuitable at low SNR.

The intermediate range presents a different balance. Here, the LS estimates contain useful channel information but remain noisy enough that interpolation, residual MIMO interference, and the Gaussian effective-noise approximation can limit the classical branch. The CNN can, in principle, use neighboring received samples, both pilot regions, the channel estimate, and the error variance to correct some of this residual mismatch. This produces a plausible “contextual-correction” interpretation of the 12–16 dB gain: the network does not replace the LS estimate, but may learn a nonlinear correction to the equalizer and demapper built on top of it.

At the highest tested SNR, the classical receiver returns to the lower mean BER. Between 16 and 20 dB, the classical BER falls by a factor of approximately 17.7, whereas the neural BER falls by only a factor of 6.9. A simple training-range explanation cannot account for this second crossover because 20 dB lies inside the training range associated with the checkpoint. A more plausible interpretation is an emerging bias-limited regime.

Panel (b) provides a second result that is not apparent from BER alone. Let E be the number of decoded information-bit errors in a frame containing $K = 3072$ information bits. Since a successful frame has $E = 0$,

$$\text{BER} = \frac{\mathbb{E}[E]}{K}, \quad \text{FER} = \Pr(E > 0), \quad (15)$$

and the mean erroneous-bit fraction conditioned on frame failure is

$$\mathbb{E}\left[\frac{E}{K} \mid E > 0\right] = \frac{\text{BER}}{\text{FER}}. \quad (16)$$

At 8 dB, the neural receiver has a higher BER but a lower FER. Equation (16) shows that approximately 18.7% of the bits are wrong in a failed neural frame, compared with 16.1% in a failed classical frame. At 10 dB, the corresponding values are approximately 15.4% and 12.8%. The neural receiver therefore moves more frames into the zero-error state, but its remaining failed frames contain more errors.

This failure distribution suggests that the neural receiver is more polarized: it completely recovers some frames that defeat the classical receiver, but it may collapse more severely on a smaller set of difficult channel realizations. One possible cause is sensitivity to deep fades or poorly conditioned 2×2 channel matrices. A second possibility concerns the decoder interface. Bitwise cross-entropy does not directly optimize frame recovery or the behavior of a finite-iteration belief-propagation decoder. A small number of incorrect but overconfident logits can be more damaging to that decoder than a larger number of weak, uncertain logits.

The learned branch appears to trade the broad stability of the analytical receiver for a gain near the decoding waterfall. It produces more error-free frames in that region, but its failed frames remain more severe and its high-SNR BER decreases more slowly.

B. Receiver Ordering Across Channel Environments

We next test whether the SNR-dependent ordering in the reference environment persists when the TDL profile, delay spread, and terminal speed change. SNR therefore appears to be the main variable controlling receiver ordering, while channel conditions become more important near the high-SNR crossover.

Figure 3(b) shows that FER changes ordering earlier than BER. The failure-severity measure in (16) gives the same interpretation as in the reference condition: at 8 dB, the neural receiver has the larger BER-to-FER ratio in all 45 environments, and at 10 dB it has the larger ratio in 44 environments. The neural branch therefore moves more frames into the zero-error

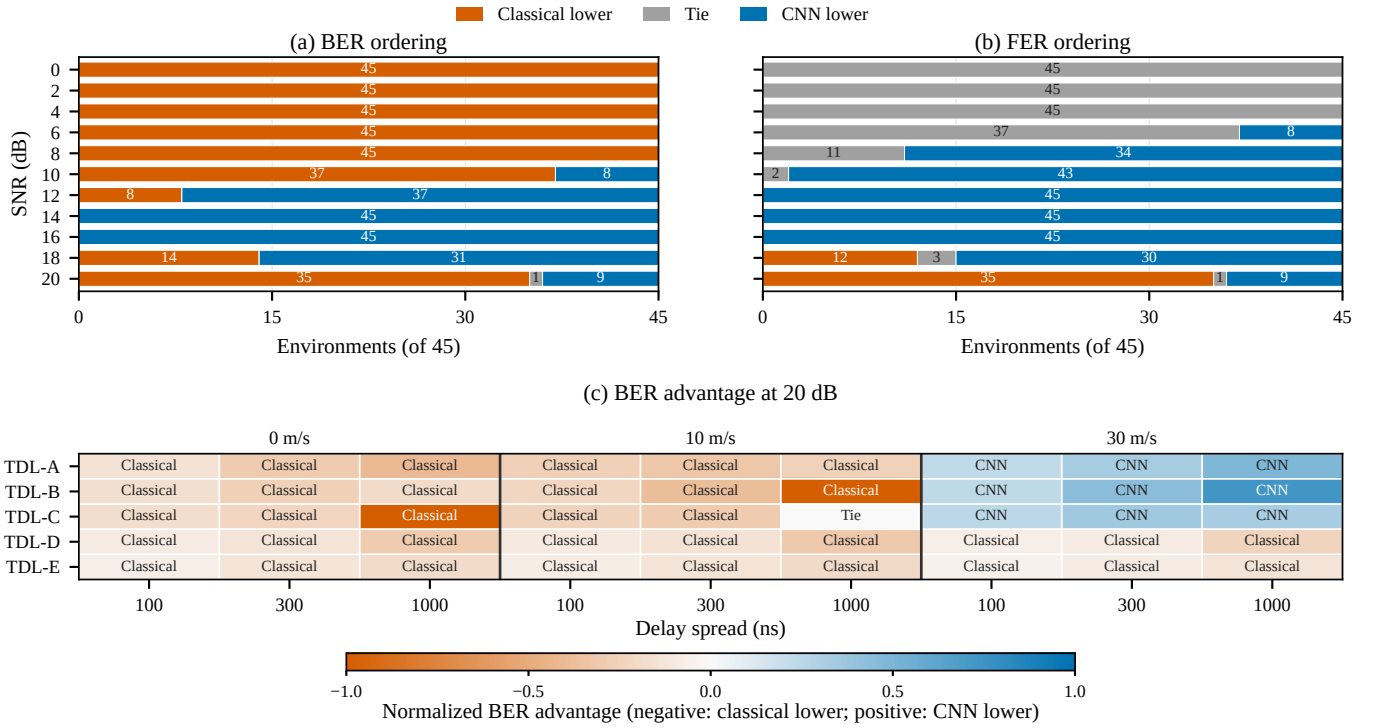


Fig. 3. Receiver ordering over the 45-environment matrix. (a), (b) Stacked bars partition each SNR into classical-lower, tied, and CNN-lower mean BER/FER cases. (c) Each cell is one TDL-profile, speed, and delay-spread combination at 20 dB. Color shows the normalized BER advantage $A = (\text{BER}_{\text{classical}} - \text{BER}_{\text{CNN}}) / (\text{BER}_{\text{classical}} + \text{BER}_{\text{CNN}})$: $A > 0$ (blue) favors the CNN and $A < 0$ (orange) favors the classical receiver; $A = 0$ when both means are zero. The panels compare three-seed means and do not establish statistical significance.

state around the decoding transition, while the frames that remain unsuccessful contain more erroneous bits.

Figure 3(c) resolves the nine neural-lower BER cases at 20 dB into a specific subset. At 30 m s^{-1} , the maximum Doppler frequency is approximately 350 Hz. Across the approximately 0.75 ms separation between the two pilot symbols, this corresponds to a maximum Doppler phase scale near 95° . One possible explanation is that this condition places greater pressure on linear time interpolation and creates more opportunity for the neural branch to use its full-grid context. Both receivers share the same LS estimates, however, and the experiment contains no interpolation-error or internal-feature measurements. The observed subset therefore does not establish that the CNN learned Doppler tracking.

The absence of neural-lower 20 dB cases under TDL-D and TDL-E shows that mobility alone does not explain the high-SNR result. Interpretation of the 1,000 ns cases is limited by the $3 \mu\text{s}$ channel window, which can truncate late channel energy.

V. CONCLUSION

This work compared an LS-assisted full-grid CNN with an implemented LS/interpolation/LMMSE/APP receiver in a coded 2×2 MIMO-OFDM link. The CNN performs best in the middle-SNR region, but its advantage weakens at high SNR. This suggests that the network is useful when estimation errors and noise are both significant, but it does not remove

the need for a model-based receiver. In particular, the classical receiver continues to benefit from cleaner observations, whereas the CNN appears to retain errors that cannot be removed by increasing SNR alone.

The BER and FER results also describe two different error behaviors. Near the decoder transition, the CNN produces more error-free frames, but the frames that still fail contain more erroneous bits. The CNN therefore appears to concentrate errors into a smaller number of badly decoded frames rather than reducing every frame by the same amount. This may be useful when complete frame recovery matters more than the average number of bit errors, but it also makes the remaining failures more severe.

The high-speed cases suggest that full-grid processing may help when linear time interpolation becomes inaccurate. This explanation is plausible, but the present results cannot confirm it. A useful next step is to compare both receivers with perfect channel knowledge and with a stronger interpolation method. That comparison would show whether the CNN is mainly correcting channel-estimation errors or learning behavior that is missing from the classical detector. Training with more high-SNR and high-mobility samples may also reduce the remaining high-SNR errors and clarify whether the observed behavior comes from the training data or from the CNN itself.

REFERENCES

- [1] S. M. Kay, *Fundamentals of Statistical Signal Processing: Estimation Theory*. Upper Saddle River, NJ, USA: Prentice Hall, 1993.

- [2] D. Tse and P. Viswanath, *Fundamentals of Wireless Communication*. Cambridge, U.K.: Cambridge University Press, 2005.
- [3] S. Coleri, M. Ergen, A. Puri, and A. Bahai, "Channel estimation techniques based on pilot arrangement in OFDM systems," *IEEE Transactions on Broadcasting*, vol. 48, no. 3, pp. 223–229, 2002.
- [4] M. Honkala, D. Korpi, and J. M. J. Huttunen, "DeepRx: Fully convolutional deep learning receiver," *IEEE Transactions on Wireless Communications*, vol. 20, no. 6, pp. 3925–3940, 2021.
- [5] D. Korpi, M. Honkala, J. M. J. Huttunen, and V. Starck, "DeepRx MIMO: Convolutional MIMO detection with learned multiplicative transformations," in *ICC 2021 – IEEE International Conference on Communications*. IEEE, 2021, pp. 1–7.
- [6] S. Cammerer, F. Ait Aoudia, J. Hoydis, A. Oeldemann, A. Roessler, T. Mayer, and A. Keller, "A neural receiver for 5G NR multi-user MIMO," in *2023 IEEE Globecom Workshops (GC Wkshps)*. IEEE, 2023, pp. 329–334.
- [7] R. Wiesmayr, S. Cammerer, F. Ait Aoudia, J. Hoydis, J. Zakrzewski, and A. Keller, "Design of a standard-compliant real-time neural receiver for 5G NR," in *2025 IEEE International Conference on Machine Learning for Communication and Networking (ICMLCN)*. IEEE, 2025, pp. 1–6.
- [8] M. Abdollahpour, M. Bertuletti, Y. Zhang, Y. Li, L. Benini, and A. Vanelli-Coralli, "A compute & memory efficient model-driven neural 5G receiver for edge AI-assisted RAN," in *GLOBECOM 2025 – 2025 IEEE Global Communications Conference*. IEEE, 2025, pp. 5248–5253.
- [9] M. Shang, Y. Zhang, L. Pang, Y. Ren, Y. Chen, and J. Li, "A design of transfer learning receiver for MIMO-OFDM systems," *IEEE Communications Letters*, vol. 28, no. 7, pp. 1564–1568, 2024.
- [10] 3GPP, "Study on channel model for frequencies from 0.5 to 100 GHz," 3rd Generation Partnership Project (3GPP), Technical Report (TR) 38.901, 2020.
- [11] J. Hoydis, S. Cammerer, F. Ait Aoudia, A. Vem, N. Binder, G. Marcus, and A. Keller, "Sionna: An open-source library for next-generation physical layer research," *arXiv preprint arXiv:2203.11854*, 2022.
- [12] 3GPP, "NR; multiplexing and channel coding," 3rd Generation Partnership Project (3GPP), Technical Specification (TS) 38.212, 2021.
- [13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [14] F. Yu and V. Koltun, "Multi-scale context aggregation by dilated convolutions," *arXiv preprint arXiv:1511.07122*, 2015.